HTML Masterclass — From Zero to Practical Web Development

1. First, understand how a website works

Imagine you are building a house.

Zidane kuće - Zidarski radovi - Zidanje kuća - Zidanje ciglom i blokom

HTML — The structure

HTML stands for HyperText Markup Language. It tells the browser what content exists and how it is organized: headings, paragraphs, images, buttons, tables and forms.

How to Spruce Up Home's Curb Appeal for a Street View?

CSS — The appearance

CSS stands for Cascading Style Sheets. It controls colors, fonts, sizes, spacing, layouts and how a website looks.

Close up of hand turning on lights at home.

JavaScript — The behavior

JavaScript makes a website interactive. For example, it can validate a form, update a shopping cart, open a menu or calculate a total.

Remember this:

- HTML = Structure
- CSS = Style
- JavaScript = Behavior

For example, in your SmartCart project, HTML can display product names and prices, CSS can style the product cards, and JavaScript can handle adding products to a shopping cart.

2. What happens when you open a website?

Your browser

Chrome, Edge or Firefox

HTML + CSS + JavaScript

The browser reads and processes the code

The finished webpage

Text, images, links, buttons and forms

HTML is a markup language, not a programming language like JavaScript. It describes the meaning and structure of content rather than writing general-purpose logic.

For example, when you write an HTML heading, the browser understands that the content is a heading.

```
<h1>Welcome to SmartCart</h1>
<p>Buy useful products online.</p>
```

The browser displays a large heading followed by a paragraph.

3. Set up your HTML workspace in VS Code

You already use VS Code, so you can practise HTML directly on your computer.

1. Open VS Code.
2. Create a folder called `HTML-Masterclass`.
3. Inside it, create a file named `index.html`.
4. Type `!` in the file and press `Tab` or `Enter` if your Emmet abbreviation suggestions are enabled. VS Code should generate the HTML starter structure.
5. Save the file and open it in Chrome. You can also use a browser-preview extension for a faster workflow.

Now understand the most important structure in HTML.

4. HTML boilerplate — the basic structure

Every normal HTML page needs a basic structure. This is often called the boilerplate.

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>My First Website</title>
</head>
<body>
   <h1>Hello, Saleem!</h1>
   <p>I am learning HTML.</p>
</body>
</html>
```

Let's understand every line.

Code	Simple meaning
`<!DOCTYPE html>`	Tells the browser to use modern HTML standards.
`<html lang="en">`	The root element of the page; the main language is English.
`<head>`	Contains information about the page.
`<meta charset="UTF-8">`	Supports a wide range of characters and symbols.
`<meta name="viewport"...>`	Helps the page fit mobile screens properly.
`<title>`	Sets the title shown in the browser tab.
`</head>`	Ends the head section.
`<body>`	Contains the visible webpage content.
`<h1>`	Creates a main heading.
`<p>`	Creates a paragraph.
`</body>`	Ends the visible content section.

`</html>`	Ends the HTML document.

Remember the difference between `head` and `body`

- `head` = information about the page.
- `body` = content shown on the page.

For example, `My First Website` inside the `<title>` element appears in the browser tab, while `Hello, Saleem!` inside the `<body>` appears on the webpage.

5. What are HTML tags, elements and attributes?

These three words are important. Do not confuse them.

A. Tags

A tag tells the browser what kind of content it is dealing with.

Hello world

- `<p>` is the opening tag.
- `</p>` is the closing tag.

B. Elements

An element usually includes the opening tag, content and closing tag.

In this example, the whole thing is the paragraph element:

Hello world

C. Attributes

Attributes give extra information to an element. They are usually written inside the opening tag.

Open Google

Here:

- `a` = anchor tag, used to create a link.
- `href` = attribute that specifies the link destination.
- `https://www.google.com` = attribute value.
- `Open Google` = clickable text.

Easy memory trick: Tag = type of content; element = the complete HTML unit; attribute = extra information.

Opening tags and closing tags

Most HTML elements have both:

<h1>My Portfolio</h1>
Welcome to my website.

But some elements are void elements. They do not wrap content and do not have closing tags.

<hr>
<img src="profile.jpg" alt="My profile photo">
<input type="text">
<meta charset="UTF-8">

- `<br>` adds a line break.
- `<hr>` represents a thematic break, often displayed as a horizontal line.
- `<img>` displays an image.
- `<input>` creates an input control.
- `<meta>` provides metadata.

You do not need to write `</br>` or `</img>`.

Nesting elements

Elements can be placed inside other elements.

I am learning HTML.

Here, `<strong>` is inside `<p>`.

Close nested tags in the correct order:

I am learning HTML.

Incorrect nesting can cause unexpected results.

Are HTML tags case-sensitive?

HTML tag names are not case-sensitive, but the recommended practice is to use lowercase.

Hello

Prefer this over `<P>Hello</P>`.

Attribute values can have case-sensitive rules depending on the attribute. Use consistent lowercase HTML names and quote attribute values.

6. Headings and paragraphs

Headings organize information. HTML provides six heading levels, from `<h1>` to `<h6>`.

```
<h1>SmartCart</h1>
<h2>Electronics</h2>
<h3>Mobile Phones</h3>
<h4>Android Phones</h4>
<h5>Budget Phones</h5>
<h6>Product Notes</h6>

<p>Welcome to our online store.</p>
```

How heading levels work

h1 — Main heading

h2 — Section heading

h3 — Subsection heading

The remaining levels, h4 to h6, organize smaller sections.

Important: Use headings based on the structure of your content, not just because you want larger text. CSS controls visual size.

A page should usually have one clear main heading, although HTML does not technically forbid multiple `<h1>` elements.

Paragraphs

Use `<p>` for a paragraph.

```
<p>My name is Saleem.</p>
<p>I am learning web development.</p>
```

Each paragraph is a separate block.

Line breaks

```
<p>First line<br>Second line</p>
```

The
 moves the following text to a new line without creating a new paragraph.

Horizontal rule

Basic information
<hr>
Contact information

The <hr> element represents a thematic change between sections.

Practical task: Create a webpage with your portfolio title, an About Me heading, a paragraph, a Skills heading, and a short skills paragraph.

7. HTML comments

Comments are notes for developers. They are not displayed as normal webpage content.

<!-- This is my portfolio heading -->
<h1>My Portfolio</h1>

The browser ignores the comment when displaying the page.

You can also temporarily comment out a block of code:

<!--
This paragraph is temporarily hidden.
-->

Remember that comments are visible to people who inspect your source code. Never put passwords, API keys or secret information in comments.

8. Text formatting tags

These tags give text meaning or formatting.

Tag	Purpose	Example
	Important content	Warning
	Draws attention visually	Featured

`<em>`	Emphasized content	`<em>Very important</em>`
`<i>`	Alternate voice or text	`<i>Scientific term</i>`
`<u>`	Underlined annotation	`<u>Check this</u>`
`<mark>`	Highlighted content	`<mark>Sale</mark>`
`<small>`	Side comments or small print	`<small>Terms apply</small>`
`<del>`	Deleted content	`<del>₹999</del>`
`<ins>`	Inserted content	`<ins>₹799</ins>`
`<sub>`	Subscript	`H<sub>2</sub>O`
`<sup>`	Superscript	`x<sup>2</sup>`

Example from an online store:

<h2>Wireless Mouse</h2>
~~₹999~~ ₹599
<mark>Limited offer</mark>
<small>Terms and conditions apply.</small>

This creates a product heading, an old price, a bold new price, a highlighted offer and a small note.

`<b>` versus `<strong>`

Both can look bold in the browser by default.

- `<b>` draws attention without saying the content is especially important.
- `<strong>` indicates that the content is important.

`<i>` versus `<em>`

Both can look italic by default.

- `<i>` is for an alternate voice, term or style.
- `<em>` indicates emphasis.

Use semantic meaning when it fits. Do not choose tags only for their appearance.

9. Lists in HTML

Lists help organize information.

A. Unordered list — `<ul>`

Use it when the order does not matter.

```
<h2>My Skills</h2>

<ul>
   <li>Python</li>
   <li>SQL</li>
   <li>HTML</li>
   <li>JavaScript</li>
</ul>
```

- `<ul>` creates the unordered list.
- `<li>` creates each list item.

The browser usually displays bullet points.

B. Ordered list — `<ol>`

Use it when the order matters.

```
<h2>How to buy a product</h2>

<ol>
   <li>Open the website.</li>
   <li>Select a product.</li>
   <li>Add it to the cart.</li>
   <li>Complete checkout.</li>
```

```
</ol>
```

The browser usually displays numbers.

C. Description list — `<dl>`

Use it for terms and their descriptions.

```
<dl>
  <dt>HTML</dt>
  <dd>Defines webpage structure.</dd>

  <dt>CSS</dt>
  <dd>Controls webpage appearance.</dd>
</dl>
```

- `<dl>` = description list.
- `<dt>` = term.
- `<dd>` = description.

Practical task

Create two lists on your portfolio page:

1. Your technical skills using `<ul>`.
2. Your learning steps using `<ol>`.

10. Links — connecting webpages

Links allow users to move from one page to another or open another resource.

A. External link

```
<a href="https://www.google.com">Open Google</a>
```

When the user clicks the text, the browser opens the destination.

B. Open a link in a new tab

```
<a href="https://www.google.com" target="_blank" rel="noopener">
  Open Google in a new tab
</a>
```

- `target="_blank"` requests a new browsing context, normally a new tab.
- `rel="noopener"` prevents the opened page from accessing the original page through `window.opener`.

C. Link to another page in your project

Suppose your folder looks like this:

HTML-Masterclass/
├── index.html
└── contact.html

In `index.html`, write:

Contact Me

This is a relative URL because it points to a file using its location within your project.

D. Link to a particular section

Go to Skills

<h2 id="skills">My Skills</h2>

When the user clicks the link, the browser navigates to the element with `id="skills"`.

E. Email and telephone links

Send an email

Call me

Replace the example email and phone number with appropriate details before using them in a real website.

Common mistake: Writing `href="contact.html"` when the file is actually in another folder. The path must match the real file location.

11. Images

The `<img>` element displays an image.

<img src="profile.jpg" alt="My profile photo">

Two important attributes:

- `src` tells the browser where to find the image.
- `alt` provides a text alternative describing the image.

A. Use an image from your project folder

Folder structure:

HTML-Masterclass/
├── index.html

```
└── images/
    └── profile.jpg
```

HTML:

```
<img src="images/profile.jpg" alt="My profile photo" width="300">
```

The browser looks for `profile.jpg` inside the `images` folder.

B. Use an image from a website

```
<img
  src="https://example.com/product.jpg"
  alt="Wireless mouse"
  width="300"
>
```

This only works if the URL points to an accessible image and the remote server permits it to be loaded.

C. Why is `alt` important?

Imagine that an image cannot load, or a person uses a screen reader. Alternative text can communicate the image's meaning.

Use descriptive text for meaningful images. For a purely decorative image, use `alt=""`.

D. Width and height

```
<img
  src="images/product.jpg"
  alt="Wireless mouse"
  width="300"
  height="200"
>
```

These attributes help define the image's display dimensions. Keep the dimensions consistent with the intended aspect ratio to avoid distortion.

Real-world example: A SmartCart product card needs an image, product name, price and a link to view the product.

12. File paths — relative and absolute

Understanding paths is essential because many beginners have images and links that do not work.

Path	Meaning
`profile.jpg`	A file in the current folder
`images/profile.jpg`	A file inside the current folder's `images` folder
`../profile.jpg`	A file one folder level above
`https://example.com/photo.jpg`	An absolute web URL

Example:

```
project/
├── index.html
├── profile.jpg
└── images/
    └── laptop.jpg
```

Inside `index.html`:

```
<img src="profile.jpg" alt="Profile">
<img src="images/laptop.jpg" alt="Laptop">
```

Both paths are relative to the location of `index.html`.

An absolute URL points to a complete web address. A relative path points to a location based on the current document.

Practical debugging rule: When an image fails to load, check the filename, extension, folder and spelling. `Laptop.jpg` and `laptop.jpg` can be different filenames on some systems and web servers.

13. Tables

Tables display information arranged in rows and columns. They are useful for marks, schedules, product comparisons and reports.

```
<table>
  <thead>
    <tr>
      <th>Product</th>
      <th>Price</th>
      <th>Stock</th>
    </tr>
  </thead>

  <tbody>
    <tr>
      <td>Wireless Mouse</td>
      <td>₹599</td>
      <td>20</td>
    </tr>

    <tr>
      <td>USB Keyboard</td>
      <td>₹799</td>
      <td>15</td>
    </tr>
  </tbody>
</table>
```

Understand the tags:

Tag	Meaning
`<table>`	The complete table
`<thead>`	The header section
`<tbody>`	The main data section
`<tr>`	One table row
`<th>`	A header cell

`<td>`	A regular data cell
`<tfoot>`	An optional footer section
`<caption>`	A title or description for the table

The browser may display a plain table without borders or spacing. CSS is used to make it attractive.

A. Merge columns using `colspan`

```
<table>
  <tr>
    <th colspan="2">Student Details</th>
  </tr>
  <tr>
    <td>Name</td>
    <td>Saleem</td>
  </tr>
</table>
```

`colspan="2"` makes the header cell span two columns.

B. Merge rows using `rowspan`

```
<table>
  <tr>
    <th rowspan="2">Department</th>
    <td>Data Analytics</td>
  </tr>
  <tr>
    <td>Web Development</td>
  </tr>
</table>
```

`rowspan="2"` makes the cell span two rows.

Important table rules

- Use tables for tabular data, not to build your entire webpage layout.
- Keep the number of cells consistent with the intended rows and columns.

- Use `<th>` for header cells and consider `scope="col"` or `scope="row"` to improve accessibility.
- Use CSS for borders, spacing and responsive behavior.

Practical exercise: Create a table containing five products, their prices and their stock quantities.

14. Forms — collecting information from users

Forms are one of the most important parts of HTML. They let users enter information such as their name, email, password, feedback and search terms.

Imagine a Contact Me page in your portfolio.

```
<form action="/contact" method="post">
    <label for="name">Your Name:</label>
    <input type="text" id="name" name="name" required>

    <br><br>

    <label for="email">Your Email:</label>
    <input type="email" id="email" name="email" required>

    <br><br>

    <label for="message">Your Message:</label>
    <textarea id="message" name="message"></textarea>

    <br><br>

    <button type="submit">Send Message</button>
</form>
```

Understand the important parts

- `<form>` groups the controls that collect user information.
- `action="/contact"` specifies the URL where the form should be submitted.
- `method="post"` requests a POST submission.
- `<label>` provides a readable label for an input.
- `<input>` creates a control for entering data.
- `id` identifies an element in the page.
- `name` identifies a field in submitted form data.
- `required` makes the field mandatory under normal browser validation.
- `<textarea>` allows longer text.
- `<button type="submit">` submits the form.

Important: This HTML alone does not create a working contact system. The `/contact` endpoint must exist on a server and process the submitted data. HTML can collect the information, but it does not automatically email it to you or save it to MySQL.

Common input types

Try these controls

Name — `type="text"`

Email — `type="email"`

Date — `type="date"`Select date...

Favourite color — `type="color"`

Preferred role — a `<select>` control
Data Analyst
Frontend Developer
Full Stack Developer
Skills — checkboxes
HTML
CSS
JavaScript

Your sample form data

Name: Not entered

Email: Not entered

Date: Not selected

Colour: #2563eb

Role: Data Analyst

Skills: HTML

This is a learning demonstration. It shows how controls work; it does not submit or store data on a server.

A. Text and password inputs

<input type="text" name="username" placeholder="Enter username">
<input type="password" name="password" placeholder="Enter password">

- `text` is for ordinary text.
- `password` visually masks what the user types. It does not, by itself, encrypt or securely store a password.

B. Email, number and date inputs

```
<input type="email" name="email">
<input type="number" name="age" min="18" max="60">
<input type="date" name="birthdate">
```

These input types provide controls and browser validation appropriate to the data.

C. Radio buttons

Radio buttons normally let the user select one option from a group.

```
<p>Choose your role:</p>

<label>
  <input type="radio" name="role" value="analyst">
  Data Analyst
</label>

<label>
  <input type="radio" name="role" value="developer">
  Developer
</label>
```

Both radio buttons use the same `name`, so they belong to one group.

D. Checkboxes

Checkboxes allow independent choices.

```
<label>
  <input type="checkbox" name="skill" value="html">
  HTML
</label>

<label>
  <input type="checkbox" name="skill" value="sql">
  SQL
</label>
```

A user can select both.

E. Dropdown list

```
<label for="city">Choose a city:</label>

<select id="city" name="city">
  <option value="">Select a city</option>
  <option value="bengaluru">Bengaluru</option>
  <option value="hyderabad">Hyderabad</option>
  <option value="chennai">Chennai</option>
```

</select>

`<select>` creates the dropdown, and `<option>` creates each choice.

F. Textarea

```
<textarea name="message" rows="4" cols="30"
        placeholder="Enter your message"></textarea>
```

Use `<textarea>` for multi-line input.

G. File upload

```
<input type="file" name="resume" accept=".pdf,.doc,.docx">
```

This lets users choose a file. The server must still validate the file type, size and contents. The `accept` attribute is a hint, not a security check.

H. Useful form attributes

Attribute	Purpose
`placeholder`	Shows a hint inside an empty control
`required`	Makes a field mandatory under normal browser validation
`value`	Sets a control's initial or submitted value
`min` / `max`	Defines allowed minimum and maximum values for supported controls
`minlength` / `maxlength`	Sets text-length limits
`checked`	Initially selects a checkbox or radio button

`selected`	Initially selects an option
`disabled`	Disables a control
`readonly`	Makes supported controls uneditable while keeping their values available for form submission
`multiple`	Allows multiple values or files where supported

I. GET versus POST

- `GET` is commonly used to retrieve information, such as search results. Form values are usually included in the URL.
- `POST` sends the form data in the request body and is commonly used for submissions that create or update data.

POST is not automatically secure. Use HTTPS and validate data on the server. Never rely only on HTML validation to protect an application.

15. `<div>` and `<span>`

These two elements are used to group content, but they behave differently by default.

A. `<div>` — a block container

```
<div>
  <h2>My Profile</h2>
  <p>I am learning web development.</p>
</div>
```

A `<div>` is a generic container. It is commonly used to group sections so CSS or JavaScript can work with them.

B. `<span>` — an inline container

```
<p>My favourite language is <span>JavaScript</span>.</p>
```

A `<span>` groups a small piece of text inside a line.

Real-world comparison

Imagine a shopping website:

- `<div>` can group the complete product card.
- `<span>` can group just the product price or discount label.

Neither element gives content special meaning on its own. Use semantic elements when they describe the content better.

16. Semantic HTML — writing meaningful webpages

Semantic HTML means using elements that describe the purpose of the content.

For example, use `<nav>` for navigation links instead of using `<div>` for everything.

A webpage layout

`<header>`

Logo and introductory content

`<nav>`

Home · About · Skills · Contact

`<main>`

`<section>`

About Me

`<article>`

Project details

`<aside>`

Related information

`<footer>`

Copyright and contact links

Important semantic tags

Tag	Purpose

`<header>`	Introductory content for a page or section
`<nav>`	Navigation links
`<main>`	The main content of the page
`<section>`	A meaningful thematic section
`<article>`	Self-contained content
`<aside>`	Related or supplementary content
`<footer>`	Footer information
`<figure>`	Self-contained media or illustrative content
`<figcaption>`	Caption for a figure
`<address>`	Contact information for the relevant person or organization

Example: a portfolio page

```
<header>
  <h1>Saleem's Portfolio</h1>
</header>

<nav>
  <a href="#about">About</a>
  <a href="#projects">Projects</a>
  <a href="#contact">Contact</a>
</nav>
```

```
<main>
  <section id="about">
    <h2>About Me</h2>
    <p>I am learning web development.</p>
  </section>

  <section id="projects">
    <h2>My Projects</h2>
    <article>
      <h3>SmartCart</h3>
      <p>An online shopping project.</p>
    </article>
  </section>
</main>

<footer>
  <p>Copyright 2026</p>
</footer>
```

Why does semantic HTML matter?

- It makes code easier to understand.
- It helps assistive technologies interpret page structure.
- It can improve maintainability.
- It gives search engines useful structural information, although semantic tags alone do not guarantee better search rankings.

17. Audio in HTML

The `<audio>` element lets a webpage play audio.

```
<audio controls>
  <source src="music.mp3" type="audio/mpeg">
  Your browser does not support HTML audio.
</audio>
```

Understand the code:

- `<audio>` creates the audio player.
- `controls` displays play, pause and other available controls.
- `<source>` specifies the audio file and its media type.
- The text inside provides fallback content for browsers that do not support the element.

Your folder could look like this:

HTML-Masterclass/

```
├── index.html
└── music.mp3
```

Because the audio file is in the same folder, `src="music.mp3"` is a relative path.

You can also use attributes such as `loop` to repeat audio or `muted` to start it muted.

```
<audio controls loop>
  <source src="music.mp3" type="audio/mpeg">
</audio>
```

Browsers commonly restrict autoplay with sound. Avoid playing audio unexpectedly because it can disrupt the user.

18. Video in HTML

The `<video>` element lets you display a video player.

```
<video controls width="500">
  <source src="my-video.mp4" type="video/mp4">
  Your browser does not support HTML video.
</video>
```

Important attributes:

- `controls` — displays playback controls.
- `width` and `height` — define the display dimensions.
- `autoplay` — requests automatic playback, subject to browser rules.
- `muted` — starts the video without sound.
- `loop` — repeats the video.
- `poster` — specifies an image to display before playback.

Example:

```
<video controls width="500" poster="images/thumbnail.jpg">
  <source src="my-video.mp4" type="video/mp4">
</video>
```

Common mistake: The video appears, but it does not play. Check the file path, file format, encoding and browser support.

Also remember that autoplay can consume mobile data and annoy users. Prefer letting people choose when to play media.

19. The `<iframe>` element

An iframe displays another webpage or an embedded resource inside your webpage.

For example, a website may embed a permitted video player or a map.

```
<iframe
  src="https://www.youtube.com/embed/VIDEO_ID"
  title="HTML learning video"
  width="560"
  height="315"
  allowfullscreen>
</iframe>
```

Replace `VIDEO_ID` with the real YouTube video ID.

Important attributes:

- `src` — the URL of the embedded resource.
- `title` — describes the iframe for accessibility.
- `width` and `height` — specify its dimensions.
- `allowfullscreen` — permits fullscreen viewing where supported.

Why might an iframe fail?

Some websites block embedding for security reasons. The browser may show a refusal or a blank frame. This is controlled by the website's security settings; changing your HTML alone cannot override those restrictions.

Practical task: Embed a YouTube video using its official embed URL, then change the iframe's width and height.

20. The `<details>` and `<summary>` elements

These elements let you create a simple expandable section without JavaScript.

```
<details>
  <summary>What is HTML?</summary>
  <p>HTML defines the structure of a webpage.</p>
</details>
```

The user clicks the summary to open or close the content.

To show the content by default:

```
<details open>
  <summary>About this course</summary>
  <p>Learn HTML step by step.</p>
</details>
```

Real-world uses include FAQs, help sections and product details.

21. HTML entities and special characters

Sometimes you need to display a character that has a special meaning in HTML.

For example, `<` and `>` are used to write tags. To display them as text, you can use HTML entities.

Entity	Display
`<`	<
`>`	>
`&`	&
`"`	"
`'`	'
`©`	©
` `	Non-breaking space

Example:

```
<p>HTML uses &lt; and &gt; to write tags.</p>
<p>Copyright &copy; 2026</p>
```

The browser displays:

- HTML uses `<` and `>` to write tags.
- Copyright © 2026

For ordinary spacing and page layout, use CSS rather than inserting many ` ` entities.

22. The <head> section — metadata and page settings

The `<head>` contains information about the document, not the main visible content.

A good basic example is:

```
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">

   <title>Saleem | Portfolio</title>

   <meta name="description"
       content="Explore my projects, skills and web development journey.">

   <link rel="icon" href="images/favicon.png">
</head>
```

Understand the key elements:

- `<title>` gives the page its browser-tab title and can be used as the title shown in search results.
- `charset` declares the document's character encoding.
- `viewport` helps mobile browsers display the page at an appropriate scale.
- `description` supplies a summary that search engines may use in search results.
- `<link rel="icon">` connects a favicon, the small icon associated with a page.

Metadata can help browsers, users and search engines understand a page. It does not guarantee any particular search ranking.

23. Accessibility — making websites usable by more people

Accessibility means designing a website so people with different abilities can use it.

Start with these simple rules:

1. Use meaningful headings in a logical order.
2. Add suitable `alt` text to meaningful images.
3. Connect form labels to their inputs using matching `for` and `id` values.
4. Use buttons for actions and links for navigation.

5. Make sure interactive elements can be used with a keyboard.
6. Use semantic HTML elements where appropriate.
7. Do not communicate important information using color alone.

Example of a properly labelled input:

<label for="email">Email address</label>
<input type="email" id="email" name="email">

A label makes it clearer what the user should enter and lets the user click the label to focus the input.

A visually attractive website is not enough. Users must also be able to understand and operate it.

24. Common HTML mistakes and how to fix them

These are the mistakes you should learn to recognize quickly.

Mistake	Why it causes trouble	Correct approach
Forgetting a closing tag	Can change how content is displayed	Close elements that require closing tags
Wrong image path	Image may not load	Check the folder and filename
Using `href` on an image	`href` belongs on links, not images	Use `src` on `<img>`
Using `src` on a normal link	Does not create a normal hyperlink	Use `href` on `<a>`
Forgetting `alt`	Image information may be inaccessible	Provide meaningful alternative text

Using the same `id` repeatedly	Can cause incorrect targeting	Keep IDs unique within the page
Using ` ` for all spacing	Creates difficult-to-maintain layouts	Use CSS for spacing
Using a table for the whole layout	Makes layout harder to maintain	Use CSS layout tools
Expecting HTML to save form data	HTML does not provide server storage	Connect the form to a backend
Expecting HTML alone to make a responsive design	Responsiveness usually requires CSS	Learn responsive CSS layouts

A quick debugging example

Suppose your image does not appear.

Your code:

<img src="image/profile.jpg" alt="My profile">

But your folder is:

```
project/
├── index.html
└── images/
    └── profile.jpg
```

The path is wrong because the folder is called `images`, not `image`.

Correct code:

<img src="images/profile.jpg" alt="My profile">

When something does not work, check the exact path before rewriting the entire code.

25. Build a complete HTML portfolio page

Now let's combine the important topics into one practical project.

Create a file named `index.html` and copy this code. This project includes headings, paragraphs, links, images, lists, a table, a form, semantic elements and an FAQ.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta name="description" content="Saleem's learning portfolio and projects.">
  <title>Saleem's Portfolio</title>
</head>
<body>

  <header>
    <h1>Saleem's Portfolio</h1>
    <p>A beginner learning web development.</p>
  </header>

  <nav>
    <a href="#about">About</a> |
    <a href="#skills">Skills</a> |
    <a href="#projects">Projects</a> |
    <a href="#contact">Contact</a>
  </nav>

  <main>
    <section id="about">
      <h2>About Me</h2>
      <p>I am learning how to build websites using HTML.</p>

      <img
        src="images/profile.jpg"
        alt="My profile photo"
        width="200"
      >
    </section>

    <section id="skills">
      <h2>My Skills</h2>

      <ul>
        <li>Python</li>
        <li>SQL</li>
        <li>HTML</li>
      </ul>
```

```
</section>

<section id="projects">
    <h2>My Projects</h2>

    <article>
        <h3>SmartCart</h3>
        <p>An online shopping application.</p>
    </article>

    <table>
        <caption>My Learning Progress</caption>
        <thead>
            <tr>
                <th scope="col">Topic</th>
                <th scope="col">Status</th>
            </tr>
        </thead>
        <tbody>
            <tr>
                <td>HTML</td>
                <td>Learning</td>
            </tr>
            <tr>
                <td>CSS</td>
                <td>Upcoming</td>
            </tr>
        </tbody>
    </table>
</section>

<section id="contact">
    <h2>Contact Me</h2>

    <form action="/contact" method="post">
        <label for="name">Name:</label>
        <input type="text" id="name" name="name" required>

        <br><br>

        <label for="email">Email:</label>
        <input type="email" id="email" name="email" required>

        <br><br>

        <label for="message">Message:</label>
        <textarea id="message" name="message" required></textarea>
```

```
            <br><br>

            <button type="submit">Send</button>
        </form>
    </section>

    <section>
        <h2>Frequently Asked Questions</h2>

        <details>
            <summary>What is HTML?</summary>
            <p>HTML defines webpage structure.</p>
        </details>
    </section>
</main>

<footer>
    <p>Copyright &copy; 2026 Saleem</p>
</footer>

</body>
</html>
```

Understand what you just built

- `header` introduces your portfolio.
- `nav` provides links to different sections.
- `main` contains the main page content.
- `section` separates topics.
- `article` groups a project description.
- `ul` lists skills.
- `table` organizes learning progress.
- `form` collects contact information.
- `details` and `summary` create an expandable FAQ.
- `footer` contains copyright information.

Important: The profile image must exist at `images/profile.jpg`. The contact form will not submit successfully unless you implement the `/contact` backend endpoint. This example teaches HTML structure; it is not yet a fully styled or functional portfolio application.

26. Practice exercises — test your knowledge

Do not just read these questions. Open VS Code and write the code yourself.

Exercise 1 — Basic webpage

Create a page with a title, a heading, two paragraphs and a horizontal rule.

Exercise 2 — Skills and links

Create an unordered list of five skills, an ordered list of three learning steps and a link to an external website.

Exercise 3 — Product page

Create a product name, image, price, old price, product description and a table showing three products.

Exercise 4 — Registration form

Create fields for name, email, password, date of birth, preferred role and skills. Add labels and required fields where appropriate.

Exercise 5 — Mini portfolio

Build the portfolio project above without copying the code. Add your own sections and make sure the links work.